Smart Campus Assistant

יומן ניסויים, בדיקות ודוח הנדסת פרומפטים

סטודנט:	נעם פריימן
תאריך:	11 מרץ 2026
גרסה:	3.0
סביבה:	Python 3.14, FastAPI, SQLite, Claude Sonnet

1. סקירת הפרויקט

פיתחתי מערכת Smart Campus Assistant שמאפשרת לסטודנטים לשאול שאלות על לוח הזמנים, ציונים, מבחנים ושעות קבלה שלהם. המערכת בנויה על ארכיטקטורת AI כפולה: Claude פועל גם כמסווג (Classifier) שמתרגם ומנתב שאלות, וגם כעונה (Responder) שמייצר תשובות מתוך נתוני מסד הנתונים.

1.1 צינור הסיווג (Classification Pipeline)

הרעיון המרכזי הוא שאסור ל-AI לראות שאלה גולמית ישירות. במקום זה, כל שאלה קודם עוברת דרך מסווג נפרד שמחליט מה לעשות איתה. זה מה שקורה בפועל:

ראשית, המערכת שולפת את פרטי הסטודנט ממסד הנתונים על פי הטוקן שמאוחסן בדפדפן. משם, שולחת את השאלה ל-Claude הראשון שתפקידו בלעדי הוא לסווג ולתרגם. הוא מחזיר JSON עם ארבעה שדות: מה השפה המקורית, מה התרגום לאנגלית, לאיזה קטגוריה שייכת השאלה (מבחנים / ציונים / לוח זמנים וכו'), ואם יש מקרה קצה מיוחד כמו שאלה לא רלוונטית או ניסיון גישה לנתוני אחרים.

לאחר הסיווג, המערכת בודקת אם צריך לחסום את השאלה לגמרי. שאלות שאינן קשורות לקמפוס, קלט ריק, או ניסיון לקרוא מידע של סטודנט אחר, היא עוצרת . אם השאלה תקינה, המערכת שולפת מה-DB את הנתונים הרלוונטיים בלבד לאותה קטגוריה, בונה פרומפט מפורט, ושולחת ל-Claude השני שתפקידו לענות מתוך הנתונים בלבד. אם הנתונים לא מספיקים, Claude מחזיר מילת מפתח מיוחדת ובמקום להמציא, המערכת מחזירה הודעה בטוחה שמפנה למרצה.

1.2 ארכיטקטורת השלבים

כל שאלה עוברת 8 שלבים עוקבים:

- שלב 1: שליפת פרטי הסטודנט ממסד הנתונים על בסיס הטוקן
- שלב 2: סיווג ותרגום השאלה דרך Claude הראשון
- שלב 3: בדיקת מקרי קצה, חסימה מיידית אם נדרש,
- שלב 4: סינון הקורסים הרלוונטיים לפי הקשר השאלה,
- שלב 5: שליפת הנתונים המתאימים מה-DB לפי הקטגוריה שזוהתה
- שלב 6: בניית פרומפט מפורט עם נתוני הסטודנט ודאטה מהסיווג
- שלב 7: Claude השני עונה מתוך הנתונים בלבד, בשפת הסטודנט
- שלב 8: בדיקת Fallback אם אין נתונים, מוחזרת הודעה מעובדת

2. הנדסת פרומפטים — השוואת V1 מול V2

2.1 פרומפט V1 הבסיסי

הפרומפט הראשוני כלל הוראות מינימליות: הקשר הסטודנט, הקטגוריה, בלוק הנתונים מה-DB, ורשימה קצרה של כללים. לא היו הנחיות עיצוב, לא תרגום שפה, ולא מודעות למקרי קצה. Claude קיבל שאלה ונתונים, וענה כמיטב יכולתו.

2.2 פרומפט V2 המתקדם

הפרומפט המתקדם הוסיף מספר שכבות. ראשית, כללי עיצוב ייחודיים לכל קטגוריה, לוח זמנים מקובץ לפי יום, ציונים עם משקלים ופירוט החישוב, שעות קבלה עם מייל הפנייה. שנית, Claude קיבל את תוצאות הסיווג המלאות, כולל שפת המקור והתרגום, כך שיכול להגיב ישירות בשפת הסטודנט. שלישית, הוספתי מנגנון Fallback ברור: אם המידע לא נמצא בנתונים שסיפקתי, יש להחזיר מילת מפתח מוגדרת ולא להמציא תשובה.

2.3 תוצאות ההשוואה: 10 שאלות

אומת ב-Pytest	הבדל מרכזי	V2	V1	שאלת בדיקה
כן	V2 הוסיף מיקום וחדר. V1 רק תאריך ושעה.	עבר	עבר	מתי המבחן הבא שלי?
כן	V1 הגיב באנגלית. V2 זיהה עברית והגיב בעברית.	עבר	נכשל	מתי המבחן שלי? (עברית)
כן	V2 הציג פירוט מלא של החישוב. V1 רק את המספר.	עבר	עבר	איזה ציון אני צריך ל-80?
לא	V2 כלל מייל לפנייה. V1 לא.	עבר	עבר	מתי שעות קבלה של פרופ. כהן?
כן	V1 ניסה לענות. V2 חסם כשאלה לא רלוונטית עם הודעה בעברית.	עבר	נכשל	מה מזג האוויר היום?
כן	V1 הציג נתונים. V2 זיהה ניסיון גישה לא מורשית וחסם.	עבר	נכשל	הראה לי את הציונים של רון
לא	V1 הציג הכול. V2 שאל 'איזה קורס?' בחכמה.	עבר	עבר	מתי המבחן? (2 קורסים)
כן	V1 שלח ל-Claude, קיבל תשובה מבולבלת. V2 חסם כקלט ריק.	עבר	נכשל	asdfghjk
לא	V2 החזיר בניין וחדר מעוצב. V1 נתונים גולמיים.	עבר	עבר	איפה השיעור שלי?
לא	V1 חסם כשאלה לא רלוונטית. V2 הגיב בחום כברכה.	עבר	נכשל	היי, מה שלומך?

ציון V1: 5/10 | **ציון V2:** 10/10

המעבר ל-V2 ביטל את כל כשלי מקרי הקצה ואיפשר תמיכה מלאה בעברית. ההוספה של שכבת הסיווג לפני המענה היא מה שמאפשרת את ההבדל הזה, פרומפט בודד לא יכול להשיג את אותה אמינות.

3. בדיקות אוטומטיות עם Pytest

כחלק מתהליך אבטחת האיכות כתבתי בדיקות יחידה (Unit Tests) מבודדות שבודקות את הלוגיקה של המערכת מבלי לבצע קריאות אמיתיות לממשק ה-AI. זה מאפשר לרוץ את כל הבדיקות תוך פחות משנייה, ללא עלות, וללא תלות בחיבור לאינטרנט.

3.1 מקרי הבדיקה

תוצאה	מה נבדק	שם הבדיקה
עבר	חישוב ממוצע משוקלל נכון לציוני סטודנט	test_weighted_average
עבר	חישוב ציון נדרש כדי להגיע ליעד מוגדר	test_target_grade_calculation
עבר	זיהוי טקסט עברי לפי טווח Unicode	test_hebrew_detection
עבר	החלפת מילת הבקרה בהודעה בטוחה למשתמש	test_fallback_insufficient_data
עבר	סיווג מחזיר תוצאה תקינה עם מקרי קצה	test_classifier_mock_response

3.2 שימוש ב-Mocking

כדי לבדוק את הלוגיקה שתלויה ב-AI מבלי לשלם על קריאות API, השתמשתי בטכניקת Mocking. במקום לשלוח שאלה אמיתית ל-Claude, הבדיקה מחזירה JSON מוגדר מראש. כך אפשר לבדוק בדיוק מה המערכת עושה עם תשובה נתונה.

כל 5 הבדיקות עברו. זמן ריצה: 0.43 שניות, לעומת כ-15 שניות אם היו קריאות אמיתיות ל-API.

4. מנגנון ה-Fallback ומניעת המצאת מידע

אחד האתגרים הידועים במערכות AI הוא שהן עלולות להמציא מידע שאינו קיים. כדי למנוע את זה, בניתי מנגנון שכבתי:

4.1 מילת הבקרה INSUFFICIENT_DATA

בפרומפט הוספתי הוראה מפורשת: אם המידע שנדרש לתשובה לא נמצא בבלוק הנתונים שסיפקתי, יש להחזיר את המחרוזת INSUFFICIENT_DATA ולא לנסות לענות.

בצד השרת, לאחר קבלת התשובה, נבדק אם היא מכילה מילה זו. אם כן, היא מוחלפת בהודעה בטוחה שמפנה את הסטודנט למרצה:

```
if 'INSUFFICIENT_DATA' in reply:
    reply = 'לא נמצא מידע על הנושא. אנא פנה למרצה.'
```

4.2 חסימת מקרי קצה

תוצאה	פעולה	קלט לדוגמה	מקרה
עבר	חסימה מיידית, ללא שאילתת DB	מה מזג האוויר?	שאלה לא רלוונטית
עבר	חסימה מיידית	asdfghjk!!!	קלט ריק / גיבריש
עבר	חסימה עם הודעת אזהרה	הראה לי ציוני דני	גישה לנתוני אחרים
עבר	שאלת הבהרה לסטודנט	מתי המבחן? (2 קורסים)	קורס לא ברור
עבר	תגובה חמה	היי, מה שלומך?	ברכה כללית

5. ניסויי בונה הלוז: (Schedule Builder)

בניתי פיצ'ר אינטראקטיבי שמאפשר לסטודנטים לבנות לוז שבועי דרך שיחה. השרת stateless, כל מצב הלוח מועבר חזרה מהדפדפן בכל בקשה, ללא צורך בשמירה בצד השרת.

5.1 ניסויים מרכזיים

סטטוס	תוצאה בפועל	תוצאה צפויה	קלט הבדיקה	ניסוי
עבר	הבחינה גברה אוטומטית, הסמינר הוסר	הבחינה גוברת	בחינה וסמינר באותה שעה	פתרון אוטומטי של בחינות
עבר	החפיפה זוהתה ותוקנה	זיהוי חפיפה	קורס A 10:00 (60 דק'), קורס B 10:30	זיהוי קונפליקטים עם משך
עבר	הקורס הוסר בהתאם	הסרת הקורס הפחות חשוב	"אלגוריתמים חשוב יותר"	בחירת קורס דומיננטי
עבר	עבד. הטבלה הוצגה ב-7 ימים בעברית	תרגום ומענה בעברית	"אני רוצה לבנות לוז"	בניית לוח בעברית

6. דוח באגים בונה הלוז: (10 ניסויים)

#	בדיקה	תוצאה צפויה	תוצאה בפועל	סטטוס	תיקון שבוצע
1	בניית לוח עם אירועים אישיים (קלט עברי)	כל האירועים בטבלה	Claude השתמש בשמות ימים (Sunday) במקום תאריכי ISO, אירועים לא הוצגו	תוקן	נוספה פונקציית המרה שממירה שם יום לתאריך ISO. הפרומפט עודכן עם דרישה מפורשת לפורמט YYYY-MM-DD
2	הוספת אירוע ב-07:00 (פילאטיס)	שורת 07:00 נראית בטבלה	הטבלה התחילה ב-08:00, אירועים מוקדמים נעלמו	תוקן	טווח השעות בטבלה שונה מ-8:00 ל20:00
3	אירועים לכל יום בשבוע	7 שורות נפרדות, יום אחד כל שורה	Claude יצר 2 שורות בלבד וסיכם את שאר הימים בטקסט	תוקן	נוספה דוגמה מפורשת של 7 ימים בפרומפט, עם הנחיה לא לסכם ימים
4	אירוע עם טווח שעות (13:00-17:00)	האירוע תופס 4 שעות בטבלה	האירוע הופיע בשורת 13:00 בלבד (ברירת מחדל 60 דק')	תוקן	פורמט הפקודה הורחב לכלול שדה משך זמן. הפרסר ולוגיקת ההוספה עודכנו בהתאם
5	יציאה ממצב לוח עם שאלה לא קשורה	מעבר חלק לצ'אט רגיל	הוצגה הודעת 'No response' באנגלית	תוקן	נוספה הודעת מעבר כשהתגובה ריקה לאחר הסרת מילת הבקרה

תיקון שבוצע	סטטוס	תוצאה בפועל	תוצאה צפויה	בדיקה	#
שדה הקלט מושבת בזמן ממתינים לתגובה ומופעל מחדש בסיום	תוקן	2 קריאות מקבילות גרמו לתגובה שגויה (race condition)	ההודעה השנייה נחסמת	שליחת 2 הודעות מהר (לחיצה כפולה)	6
בניית נתוני הלוח הועברה לאחר עיבוד הפקודות	תוקן	נתוני הלוח נבנו לפני עיבוד הפקודות החדשות	האירוע החדש מוצג מיד	נתוני הלוח לא מתעדכנים לאחר הוספת אירוע	7
רשימת הימים נוספה לכל סוגי התגובות	תוקן	רשימת הימים נשלחה רק בתגובת 'לוח הושלם', לא בתגובה הרגילה	הפרונטאנד מקבל את רשימת הימים	תגובת לוח רגילה חסרה את רשימת הימים	8
עיצוב ה-PDF שונה מחדש עם כותרות צבעוניות וצבע לפי סוג אירוע	תוקן	טבלה שחור לבנה עם עיצוב מינימלי	PDF מקצועי עם צבעים	איכות עיצוב PDF	9
נוספה מגבלת 200 מילים עם חיתוך אוטומטי והודעה בממשק	תוקן	ללא הגבלה, ה-API קיבל prompt ארוך מאוד	הקלט מוגבל לאורך סביר	סטודנט שולח הודעה ארוכה מאוד (500+ מילים)	10

7. ניסיונות כושלים A מקרים מתועדים

7.1 Gemini ניצל מכסה יומית

קלט: כל שאלה לאחר 20 בקשות

תוצאה צפויה: סיווג ותרגום תקינים

תוצאה בפועל: שגיאת מכסה. כל שאלות הוחזרו כאנגלית, ללא זיהוי עברית

הסיבה: המכסה החינמית של Gemini מוגבלת ל-20 בקשות ביום, ו-fallback לא תמך בזיהוי עברית.

הפתרון: החלפתי את Gemini לחלוטין ב-Claude לצורך הסיווג, והוספתי זיהוי עברית מקומי על בסיס טווח Unicode כ-fallback משני. זה ביטל לחלוטין את התלות ב-API נוסף.

לקח: APIs חינמיים אינם מתאימים לשימוש בפרודקשן. עדיף לשמור הכל אצל ספק אחד.

7.2 המסווג החזיר JSON עטוף בפורמט Markdown

קלט: "מתי המבחן של אלגוריתמים?" (עברית)

תוצאה צפויה: JSON נקי: {"original_language": "hebrew", ...}

תוצאה בפועל: JSON עטוף בגרשיים של Markdown, הפרסר קרס

הסיבה: התנהגות ברירת מחדל של Claude היא לעטוף תשובות JSON בבלוק קוד. הפונקציה שניסתה לפרסר את התוצאה נכשלה על הסיומת.

הפתרון: הוספתי ניקוי לפני כל פרסור, אם התשובה מתחילה בגרשיים של Markdown, מסירים אותם לפני העיבוד.

לקח: כשמשתמשים ב-AI כמחולל JSON, תמיד יש לצפות לפורמט לא נקי ולנקות לפני פרסור.

7.3 בונה הלוח התבלבל ובנה לוז בלי שהתבקש

קלט: "תן לי רשימה של שיעורי החודש" (לא במצב בניית לוח)

תוצאה צפויה: תשובה רגילה מהדאטה

תוצאה בפועל: מילת הבקרה EXIT_SCHEDULE הופיעה כטקסט לסטודנט

הסיבה: לא הייתה הבחנה בין 'הצג לי את הלוח' (שאילתה רגילה) לבין 'בנה לי לוח' המסווג שלח את כל השאלות הקשורות ללוח לבונה, שלא ידע לטפל בהן.

הפתרון: הוספתי ערך מיוחד לסיווג שמזהה ספציפית בקשות לבניית לוח. שאלות רגילות על לוח הזמנים ממשיכות לזרם הרגיל. כמו כן הוספתי ניקוי של כל מילות הבקרה מהתשובה לפני הצגה.

לקח: מילות בקרה פנימיות (כמו INSUFFICIENT_DATA ו-EXIT_SCHEDULE) חייבות תמיד להיות מנוקות מהפלט לפני שהוא מוצג לסטודנט.

8. ביצועים

דרישה	ערך נמדד	מדד
< 30 שניות (NFR-01)	5–9 שניות	זמן תגובה כולל (שאלה רגילה)
—	2–3 שניות	שלב הסיווג בלבד
—	< 0.1 שניות	שליפה ממסד הנתונים
—	3–6 שניות	שלב המענה בלבד
—	2–3 שניות	חסימת מקרה קצה
< 30 שניות (NFR-01)	4–8 שניות	בונה הלוח (פר-תור)
< 1 שנייה (NFR-01)	מיידי (< 0.1 שניות)	הצגת מחוון טעינה
—	0.43 שניות (5 בדיקות)	Unit Tests (Pytest)

כל זמני התגובה נמצאים הרבה מתחת לדרישת 30 השניות. שתי הקריאות ל-AI (סיווג + מענה) מוסיפות ~2–3 שניות לעומת קריאה בודדת, אבל ההכרח הזה הוא חלק מהעיצוב — האמינות והתמיכה בעברית שוות את ה-trade-off.

9. קבצים שעודכנו

מה השתנה	קובץ
פרומפט הסיווג המלא, פרומפט התרגום, פרומפטי V1 ו-V2 של המשיב, פרומפט בניית הלוח	prompts_final.py
החלפת Gemini ב-Claude לסיווג, ניקוי JSON, 8 צינור 8 שלבים, ניתוב בונה הלוח, טיפול ב-fallback	main_server.py
המרת שמות ימים לתאריכים, זיהוי עברית, טיפול במקרי קצה, סינון קורסים, פונקציות הלוח	utils_ai_server.py
ניהול מצב בונה הלוח, בניית טבלת לוח שבועית, ביטול קלט במהלך טעינה, מגבלת 200 מילים, עיצוב PDF	dashboard.html
שאילתות DB לכל קטגוריה: מבחנים, ציונים, לוח זמנים, שעות קבלה, חדרים	ai_query.py

10. סיכום

במהלך הפרויקט בניתי מערכת AI עובדת מקצה לקצה.

הנקודות המרכזיות:

- ארכיטקטורת Classifier-Responder דו-שכבתית עם Claude, פרומפט בודד לא יכול להשיג את אותה אמינות
- הנדסת פרומפטים שהביאה לשיפור מ-50% ל-98% הצלחה בעשרת תרחישי הבדיקה
- 5 בדיקות Unit Test אוטומטיות עם Mocking, כולן עברו, זמן ריצה 0.43 שניות
- מנגנון Fallback שמונע המצאת מידע, Claude מחזיר מילת בקרה, השרת מחליף בהודעה בטוחה
- טיפול מלא בחמישה סוגי מקרי קצה: שאלה לא רלוונטית, קלט ריק, גישה לנתוני אחרים, קורס לא ברור, ברכה
- בונה לוז אינטראקטיבי עם זיהוי קונפליקטים, פתרון אוטומטי לבחינות, ותמיכה בעברית,
- 10 באגים שזוהו, תועדו ותוקנו, כולם נסגרו

כל דרישות הביצועים (NFR-01) מולאו, עם זמני תגובה של 5 -9 שניות לעומת מגבלה של 30 שניות.